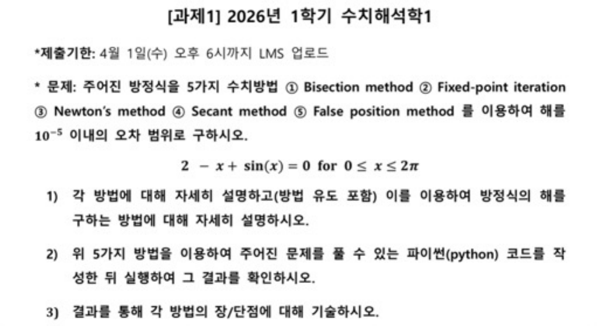


[과제1] 2026년 1학기 수치해석학1

***제출기한:** 4월 1일(수) 오후 6시까지 LMS 업로드

*** 문제: 주어진 방정식을 5가지 수치방법 ① Bisection method ② Fixed-point iteration ③ Newton's method ④ Secant method ⑤ False position method 를 이용하여 해를 10^{-5} 이내의 오차 범위로 구하시오.**

$$2 - x + \sin(x) = 0 \text{ for } 0 \le x \le 2\pi$$

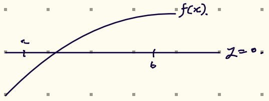
1) 각 방법에 대해 자세히 설명하고(방법 유도 포함) 이를 이용하여 방정식의 해를 구하는 방법에 대해 자세히 설명하시오.

2) 위 5가지 방법을 이용하여 주어진 문제를 풀 수 있는 파이썬(python) 코드를 작성한 뒤 실행하여 그 결과를 확인하시오.

3) 결과를 통해 각 방법의 장/단점에 대해 기술하시오.

① Bisection Method.

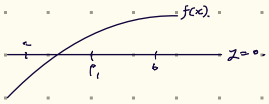
* 중간값 정리.



함수 $f(x)$가 닫힌 구간 $[a,b]$ 에서 연속이고.

$f(a)f(b)<0$ 이면 $f(x)=0$을 만족하는

근이 적어도 하나 존재.



중간값 정리에 의해 ($f(a)f(b)<0$) 해가 있는 구간 $[a,b]$의

중점 p_1에 대해 $\left(p_1 = a + \frac{(b-a)}{2}\right)$.

$$\begin{cases} f(p_1)=0 \rightarrow p_1=p, \ p_1\text{이 해.} \\ f(a)\cdot f(p_1)<0 \rightarrow [a,p_1] \text{ 구간 내에 } p \text{ 존재.} \\ f(p_1)\cdot f(b)<0 \rightarrow [p_1,b] \text{ 구간 내에 } p \text{ 존재.} \end{cases}$$

다음 과정에 대해 해가 존재하는 구간에 대해 다시 위 과정을 반복 수행하여

$[a,p_1]$ 구간 내에 해 p 존재 시.

중점 $p_2 = a + \frac{(p_1-a)}{2}$ 와 같이 반복하여 간격을 좁혀가며 구간 폭이

허용 오차(eps) 보다 작은 경우 반복을 멈추어 근사 해를 구해낸다.

장·단점, 위와 같이 구간을 좁혀나가기에 (반씩) 반드시 수렴해가며 오차의 범위가 분명하다는 장점이 존재하는 반면. 반씩 줄여가기에 수렴 속도가 느리다.

② Fixed point Iteration.

*

Fixed point의 정의.

연속하는 함수 $g(x)$에 대하여 방정식 $g(x)=x$를 만족하는

해. → g의 Fixed point.

초기값 x_0에 대하여

$x_n = g(x_{n-1})$, $n \ge 1$, 다음의 반복식을 통해

$x_1, x_2, \cdots x_n$으로 근사해를 구해 낼 수 있다.

이때. x_n이 수렴하는 수열이고. $\lim_{n\to\infty} x_n = \alpha$, $n \ge 1$ 이면

함수 g가 연속이기에 $\lim_{n\to\infty} g(x_n) = g(\alpha)$ 으로

$\alpha = g(\alpha)$가 되기에 극한값 α는 g의 고정점이 된다.

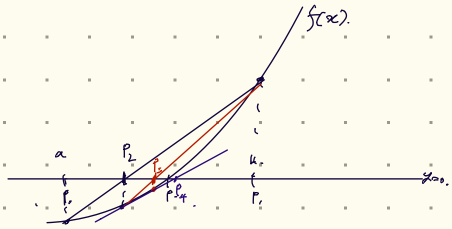
장단점

구간을 무조건 절반으로 나누는 이분법에 비해 빠르게

근사할 수 있지만 초기값 선택에 따라 해를 못 구할 수도

발산하는 경우도 발생한다. (구간과, 초기값 중요)

Secant method.



$f(p)=0$인 p가 구간 $[a,b]$ 내에 존재할 때.

$p_0 = a$, $p_1 = b$ 인 두 점 $(p_1, f(p_1))$, $(p_2, f(p_2))$ 에 대하여

$$y = \frac{f(p_1) - f(p_0)}{p_1 - p_0} \cdot (x - p_1) + f(p_1)$$ 에 대하여 $y=0$ 일 때의

해 p_2를 구하며 이를 반복하며 p_n의 근사해를 구하는 방식.

$$0 = \frac{f(p_{n-2}) - f(p_{n-1})}{(p_{n-1} - p_{n-2})} \cdot (p_n - p_{n-1}) + f(p_{n-1}).$$

$$p_n = p_{n-1} + \frac{p_{n-1} - p_{n-2}}{f(p_{n-2}) - f(p_{n-1})} f(p_{n-1}).$$

장·단점.

해당 방식의 경우 위의 방식과 같이 초기값에 따라

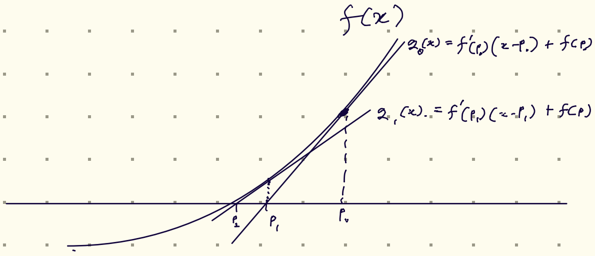
발산 할 수 있기에 적절한 초기값의 선택 필요.

또한 해당 방식의 경우 접선의 기울기가 아닌 두 점을 통한 할선을

구하여 근사해를 구하는 방식으로. 구간 내에서 미분되지 않는

경우에도 해를 구할 수 있다.

Newton's Method.


$f(x)$
$\ell_0(x) = f'(p_0)(x-p_0) + f(p_0)$
$\ell_1(x) = f'(p_1)(x-p_1) + f(p_1)$
p_2
p_1
p_0

함수 f 에 대해 구간 $[a, b]$ 에서 f''이 연속이고, $f \in C^2[a, b]$
$f(p) = 0$인 p가 구간 $[a, b]$ 에서 존재 할때 $p \in [a, b]$
초기 근사 값 $p_0 \in [a, b]$의 점 $(p_0, f(p_0))$ 에서의 접선의 방정식이
x 축과 만나는 점을 p_1 으로 하여 근사해를 구해나가는 방식.

점 $(p_n, f(p_n))$의 접선의 방정식 $\ell_n(x)$ 는

$$\ell_n(x) = f'(p_n)\cdot(x - p_n) + f(p_n)$$ 이다.

이때. $\ell_n(p_{n+1}) = 0$ 임으로.

$$\ell_n(p_{n+1}) = f'(p_n)(p_{n+1} - p_n) + f(p_n) = 0$$

$$p_{n+1} - p_n + \frac{f(p_n)}{f'(p_n)} = 0$$

$$p_{n+1} = p_n - \frac{f(p_n)}{f'(p_n)} = 0.$$

다음과 같이 반복하며 근사해를 구해나간다.

이때. $f(p) = 0$ 인 p로 근사해 p_n이 수렴한다면 (가정).

$$f(p) = f(p_n) + (p - p_n) f'(p_n) + \underline{\frac{(p-p_n)^2}{2!} f''(c_n)}$$

↳ c_n은 p와 p_n 사이에 존재.

테일러 전개를 통해 위와 같이 나타낼 수 있다

이때. $f'(p_n)$으로 양변을 나누면.

$$0 = \frac{f(p_n)}{f'(p_n)} + (p - p_n) + \frac{(p-p_n)^2}{2!}\frac{f''(c)}{f'(p_n)}.$$

$\frac{f(p_n)}{f'(p_n)} = p_n - p_{n+1}$ 임을 통하여.

$$0 = p_n - p_{n+1} + p - p_n + (p-p_n)^2 \cdot \frac{f''(c)}{2f'(p_n)}$$

$$= p - p_{n+1} + (p-p_n)^2 \cdot \frac{f''(c)}{2f'(p_n)}$$

즉 $|p - p_n| = E_n$ 라하고.

$\frac{f''(c)}{2f'(p_n)}$ 를 상수 k 라한다면.

$$-(p - p_{n+1}) = (p - p_n)^2 \frac{f''(c)}{2f'(p_n)}$$

는

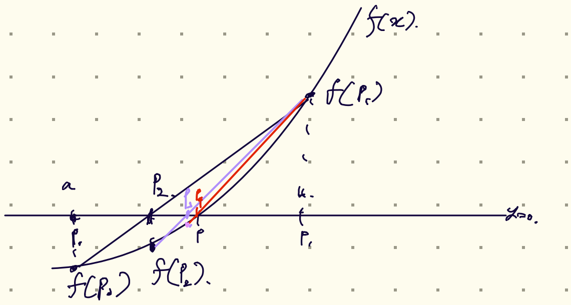
$$E_{n+1} = (E_n)^2 k.$$ 가 되어진다.

장·단점. 즉 적절한 초기값을 통해 $|E_n| < 1$ 라면
다음오차는 2제곱에 비례하게 줄어들며 수렴하게된다.

즉 적절한 초기값을 잡지 못한다면
발산하거나 (오차가 계속커짐), 느리게 수렴하게 되어는

$|E_n| = 10^{-2}$ 오차가 되어 $|E_{n+1}| = 10^{-4}$ 로 적절한 초기값을 잡는다면
매우 빠르게 해를 구할 수 있다.

False-position.



$f(p) = 0$ 인 p가 구간 $[a, b]$ 내에 존재할 때. $f(a)\cdot f(b) < 0$

$p_0 = a$, $p_1 = b$ 인 두점 $(p_0, f(p_0))$, $(p_1, f(p_1))$ 에 대하여

$$① \; p_n = p_{n-1} + \frac{p_{n-1} - p_{n-2}}{f(p_{n-2}) - f(p_{n-1})} f(p_{n-1}).$$

을 통해. p_2 를 도출.

이후. $f(p_2)$ 에 대해서

$f(p_0)\cdot f(p_2) < 0$ 인경우

$\underline{p_0}$, $\underline{p_2}$, $f(p_0)$, $f(p_2)$ 에 대해 ① 적용하는

p_3 를 도출. $E = |p_0 - p_2|$

$f(p_1)\cdot f(p_2) < 0$ 인경우.

p_1, p_2, $f(p_1)$, $f(p_2)$ 에 대해서 ① 적용하는.

p_3 를 도출 $E = |p_1 - p_2|$.

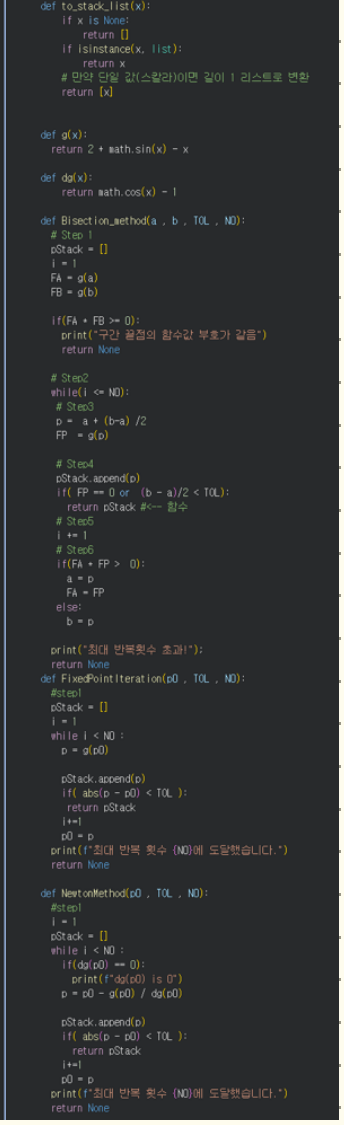
E가 ToL 보다 작을때 까지 반복 하여 근사해 p_n을 구하는 방식.

장·단점.

secant method와 마찬가지로. 미분이 안되는 경우에도 적용할 수 있다.

Bisection 의 경우 일정 비율로만 줄어들지만 false position 더크게 줄어들 수 있음. 단 구간이 한쪽으로 치우칠 경우 더 작게 줄어들수 있다.

Bisection과 같이 초기 값, 두점 $(a, f(a))$, $(b, f(b))$ 에 대해
$f(a)\cdot f(b) < 0$ 에만 가능하다.

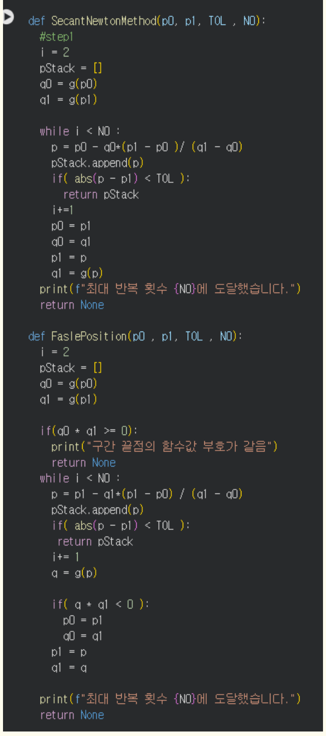


```
def to_stack_list(x):
    if x is None:
        return []
    if isinstance(x, list):
        return x
    # 만약 단일 값(스칼라)이면 길이 1 리스트로 변환
    return [x]


def g(x):
  return 2 + math.sin(x) - x

def dg(x):
    return math.cos(x) - 1

def Bisection_method(a , b , TOL , NO):
  # Step 1
  pStack = []
  i = 1
  FA = g(a)
  FB = g(b)

  if(FA * FB >= 0):
    print("구간 끝점의 함수값 부호가 같음")
    return None

  # Step2
  while(i <= NO):
   # Step3
   p =  a + (b-a) /2
   FP  = g(p)

   # Step4
   pStack.append(p)
   if( FP == 0 or  (b - a)/2 < TOL):
     return pStack #<-- 함수
   # Step5
   i += 1
   # Step6
   if(FA * FP >  0):
     a = p
     FA = FP
   else:
     b = p

  print("최대 반복횟수 초과!");
  return None
def FixedPointIteration(p0 , TOL , NO):
  #step1
  pStack = []
  i = 1
  while i < NO :
    p = g(p0)

    pStack.append(p)
    if( abs(p - p0) < TOL ):
     return pStack
    i+=1
    p0 = p
  print(f"최대 반복 횟수 {NO}에 도달했습니다.")
  return None

def NewtonMethod(p0 , TOL , NO):
  #step1
  i = 1
  pStack = []
  while i < NO :
    if(dg(p0) == 0):
      print(f"dg(p0) is 0")
    p = p0 - g(p0) / dg(p0)

    pStack.append(p)
    if( abs(p - p0) < TOL ):
      return pStack
    i+=1
    p0 = p
  print(f"최대 반복 횟수 {NO}에 도달했습니다.")
  return None
```

Bisection _method.

Fixed Point Iteration.

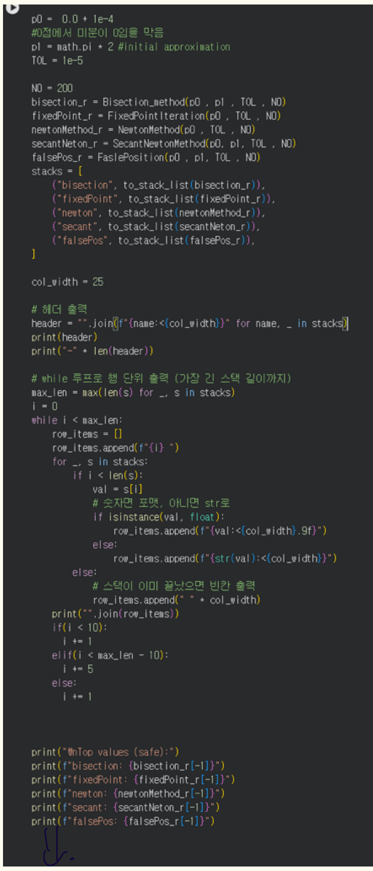
·Neton's Method.



```
def SecantNewtonMethod(p0, p1, TOL , NO):
  #step1
  i = 2
  pStack = []
  q0 = g(p0)
  q1 = g(p1)

  while i < NO :
    p = p0 - q0*(p1 - p0 )/ (q1 - q0)
    pStack.append(p)
    if( abs(p - p1) < TOL ):
      return pStack
    i+=1
    p0 = p1
    q0 = q1
    p1 = p
    q1 = g(p)
  print(f"최대 반복 횟수 {NO}에 도달했습니다.")
  return None

def FaslePosition(p0 , p1, TOL , NO):
  i = 2
  pStack = []
  q0 = g(p0)
  q1 = g(p1)

  if(q0 * q1 >= 0):
    print("구간 끝점의 함수값 부호가 같음")
    return None
  while i < NO :
    p = p1 - q1*(p1 - p0) / (q1 - q0)
    pStack.append(p)
    if( abs(p - p1) < TOL ):
     return pStack
    i+= 1
    q = g(p)

    if( q * q1 < 0 ):
      p0 = p1
      q0 = q1
    p1 = p
    q1 = q

  print(f"최대 반복 횟수 {NO}에 도달했습니다.")
  return None
```

Secant Neton Method

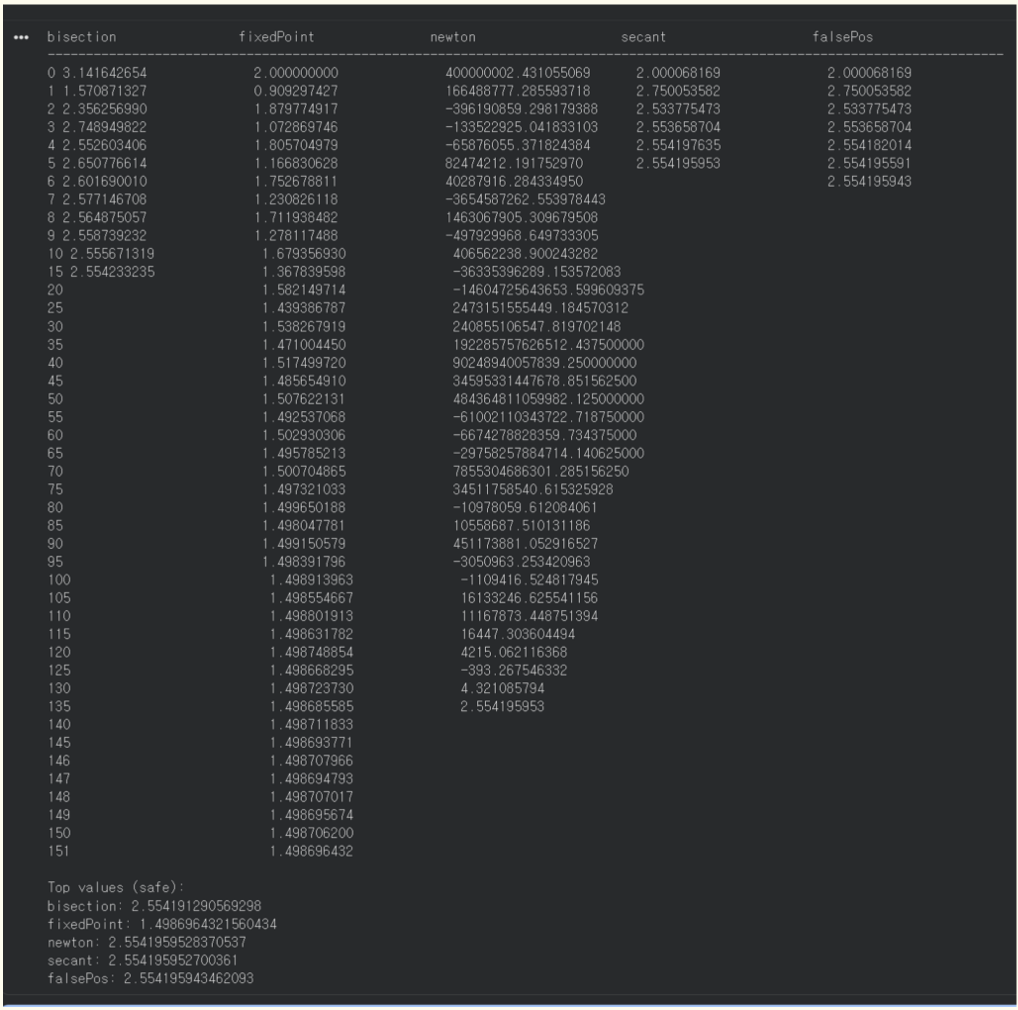
False _ Position



```
p0 =  0.0 + 1e-4
#0점에서 미분이 0임을 막음
p1 = math.pi * 2 #initial approximation
TOL = 1e-5

NO = 200
bisection_r = Bisection_method(p0 , p1 , TOL , NO)
fixedPoint_r = FixedPointIteration(p0 , TOL , NO)
newtonMethod_r = NewtonMethod(p0 , TOL , NO)
secantNeton_r = SecantNewtonMethod(p0, p1, TOL , NO)
falsePos_r = FaslePosition(p0 , p1, TOL , NO)
stacks = [
    ("bisection", to_stack_list(bisection_r)),
    ("fixedPoint", to_stack_list(fixedPoint_r)),
    ("newton", to_stack_list(newtonMethod_r)),
    ("secant", to_stack_list(secantNeton_r)),
    ("falsePos", to_stack_list(falsePos_r)),
]

col_width = 25

# 헤더 출력
header = "".join(f"{name:<{col_width}}" for name, _ in stacks)
print(header)
print("-" * len(header))

# while 루프로 행 단위 출력 (가장 긴 스택 길이까지)
max_len = max(len(s) for _, s in stacks)
i = 0
while i < max_len:
    row_items = []
    row_items.append(f"{i} ")
    for _, s in stacks:
        if i < len(s):
            val = s[i]
            # 숫자면 포맷, 아니면 str로
            if isinstance(val, float):
                row_items.append(f"{val:<{col_width}.9f}")
            else:
                row_items.append(f"{str(val):<{col_width}}")
        else:
            # 스택이 이미 끝났으면 빈칸 출력
            row_items.append(" " * col_width)
    print("".join(row_items))
    if(i < 10):
      i += 1
    elif(i < max_len - 10):
      i += 5
    else:
      i += 1



print("₩nTop values (safe):")
print(f"bisection: {bisection_r[-1]}")
print(f"fixedPoint: {fixedPoint_r[-1]}")
print(f"newton: {newtonMethod_r[-1]}")
print(f"secant: {secantNeton_r[-1]}")
print(f"falsePos: {falsePos_r[-1]}")
```

↓.

출력 단.



```
bisection                fixedPoint               newton                   secant                   falsePos
-----------------------------------------------------------------------------------------------------------------------------
0 3.141642654              2.000000000              400000002.431055069      2.000068169              2.000068169
1 1.570871327              0.909297427              166488777.285593718      2.750053582              2.750053582
2 2.356256990              1.879774917              -396190859.298179388     2.533775473              2.533775473
3 2.748949822              1.072869746              -133522925.041833103     2.553658704              2.553658704
4 2.552603406              1.805704979              -65876055.371824384      2.554197635              2.554182014
5 2.650776614              1.166830628              82474212.191752970       2.554195953              2.554195591
6 2.601690010              1.752678811              40287916.284334950                                2.554195943
7 2.577146708              1.230826118              -3654587262.553978443
8 2.564875057              1.711938482              1463067905.309679508
9 2.558739232              1.278117488              -497929968.649733305
10 2.555671319              1.679356930              406562238.900243282
15 2.554233235              1.367839598              -36335396289.153572083
20                          1.582149714              -14604725643653.599609375
25                          1.439386787              2473151555449.184570312
30                          1.538267919              240855106547.819702148
35                          1.471004450              192285757626512.437500000
40                          1.517499720              90248940057839.250000000
45                          1.485654910              34595331447678.851562500
50                          1.507622131              484364811059982.125000000
55                          1.492537068              -61002110343722.718750000
60                          1.502930306              -6674278828359.734375000
65                          1.495785213              -29758257884714.140625000
70                          1.500704865              7855304686301.285156250
75                          1.497321033              34511758540.615325928
80                          1.499650188              -10978059.612084061
85                          1.498047781              10558687.510131186
90                          1.499150579              451173881.052916527
95                          1.498391796              -3050963.253420963
100                          1.498913963              -1109416.524817945
105                          1.498554667              16133246.625541156
110                          1.498801913              11167873.448751394
115                          1.498631782              16447.303604494
120                          1.498748854              4215.062116368
125                          1.498668295              -393.267546332
130                          1.498723730              4.321085794
135                          1.498685585              2.554195953
140                          1.498711833
145                          1.498693771
146                          1.498707966
147                          1.498694793
148                          1.498707017
149                          1.498695674
150                          1.498706200
151                          1.498696432

Top values (safe):
bisection: 2.554191290569298
fixedPoint: 1.4986964321560434
newton: 2.5541959528370537
secant: 2.554195952700361
falsePos: 2.554195943462093
```